# Adaptive Latent Reasoning: PonderNet-style Probabilistic *Halting* over Implicit Chains of Thought

**Valentino Arbelaiz Alberti**
varbelaizalberti@udesa.edu.ar

**Ana Paula Tissera**
atissera@udesa.edu.ar

**Franco Amato de Lusarreta**
famatodelusarreta@udesa.edu.ar

**Juan Cruz Giner Pulero**
jginer@udesa.edu.ar

**Naomi Couriel**
ncouriel@udesa.edu.ar

## Abstract

*Latent chain-of-thought* reasoning models replace natural-language reasoning steps with a sequence of continuous vectors, but they fix their compute budget in advance (how many latent steps $K$ to run and how many vectors $c$ represent each step) and apply it uniformly to every input: a two-step addition problem consumes the same budget as a six-step algebraic one. This work studies how to make that budget **instance-adaptive** along the Coconut $\rightarrow$ CODI $\rightarrow$ SIM-CoT line, organizing the exploration of its two axes into three studies. A first study (*upfront* prediction) verifies the premise with an oracle sweep over CODI (the optimal step budget exists, varies per instance, and is strongly imbalanced), but shows that anticipating it *before* reasoning does not beat the majority class (a classifier over frozen prompt embeddings). Adaptive *halting* attacks the $K$ axis *during* reasoning: it grafts PonderNet's probabilistic *halting* mechanism, where a *halting* head defines a termination distribution over prefixes and a KL regularizer toward a per-instance geometric prior calibrates the budget. On GSM8k with a GPT-2 backbone, evaluating on the held-out test set, the adaptive model matches the fixed-$K$ baseline accuracy ($\approx 39.5\%$) while cutting nearly half ($\sim 50\%$) of the latent steps, with strong difficulty tracking (Spearman correlation of $+0.626$); we thus characterize the **accuracy–compute frontier** that prior work, tied to a single $K$, never reported. A third study explores the $c$ axis and finds that it has almost no exploitable headroom on GSM8k-Aug (accuracy saturates at $c$=2 almost uniformly). The robust result is **compute adaptivity**: at parity accuracy, the model allocates fewer steps to easy problems and more to hard ones.

## 1 Introduction

Explicit *chain-of-thought* (CoT) improves the reasoning of language models by generating intermediate steps in text (Wei et al., 2022), but it pays a cost: each step is decoded token by token over the vocabulary, and most of those tokens exist only for fluency. *Latent reasoning* (or implicit CoT) removes that cost by performing the reasoning in a continuous space: instead of projecting the hidden state to a token and converting it back into an embedding, the model feeds its last hidden state directly as the next input embedding, producing a sequence of "continuous thoughts" that are never decoded to text (Hao et al., 2024). This family of methods (Coconut (Hao et al., 2024), CODI (Shen et al., 2025), and SIM-CoT (Wei et al., 2025)) reaches the accuracy of explicit CoT at a fraction of the inference cost.

All of them, however, share a structural limitation: the number of latent steps $K$ is a **fixed hyperparameter**, identical for every input. A two-step addition problem and a six-step algebraic one consume exactly the same reasoning budget. This wastes compute on easy inputs and may fall short on hard ones. The premise that compute should scale with difficulty (not with input size) is precisely what motivates the *adaptive computation time* literature (Graves, 2016) and, in particular, PonderNet (Banino et al., 2021), which learns a termination distribution over the steps of a recurrent computation.

The latent budget has two axes: how many reasoning steps to run ($K$) and how many vectors represent each step ($c$). We organize the work into three studies that sweep that space and converge on one reading: adaptivity cannot be read from the prompt (*upfront* prediction), nor does it live in the width of the step ($c$ axis), but rather in the *number* of steps, decided while the model reasons (adaptive *halting*). The latter, *halting* over the CODI/SIM-CoT loop, is our central contribution. Our contributions, one per study, are:

1. ***Upfront* prediction ($K$ axis).** An oracle sweep confirms the premise (the optimal bud-

get $k^*$ exists, varies per instance, and saturates), but predicting it from frozen prompt embeddings does not beat the majority class: an informative negative result (Section 4).

2. **Adaptive halting ($K$ axis, online).** We graft PonderNet's *halting* onto the CODI/SIM-CoT latent loop, with a per-instance geometric prior anchored to the number of operations, and characterize the accuracy–compute frontier that fixed $K$ cannot trace (Section 5).

3. **The $c$ axis (vectors per step).** The adaptive mechanism works, but the required $c$ is almost constant on GSM8k-Aug: a second negative that bounds the design space (Section 6).

We frame the result honestly: the accuracy gains over the fixed-$K$ baseline are within the noise. The robust and reproducible result is **compute adaptivity**: at parity accuracy, the model allocates fewer steps to easy problems and more to hard ones.

## 2 Related Work

**Implicit CoT.** **Coconut** (Hao et al., 2024) proposes reasoning in a continuous latent space by feeding the last hidden state as the next input, and observes that a single continuous thought can encode several alternative continuations at once (a BFS-like search). It is trained with a multi-stage curriculum that progressively replaces language steps with continuous thoughts, but it suffers *catastrophic forgetting* between stages and collapses as the number of latents grows. **CODI** (Shen et al., 2025) removes the curriculum with a *single-stage self-distillation*: it jointly trains a teacher task (explicit CoT) and a student (continuous thoughts) and aligns their hidden states at a single *distillation token* (the position just before the answer), with a *stop-gradient* on the teacher. It is the first implicit CoT to match explicit CoT at GPT-2 scale. **SIM-CoT** (Wei et al., 2025) diagnoses that the instability when scaling $K$ comes from the *homogenization* of the latents due to the lack of per-step supervision, and adds an *auxiliary decoder* that reconstructs the text of each reasoning step during training; the decoder is discarded at inference, so there is no added cost. It stabilizes training up to $K = 8$. In all three methods, however, $K$ remains **fixed** at inference.

**Adaptive computation.** **ACT** (Graves, 2016) introduces adaptive computation in recurrent networks through a *halting* unit, with biased gradients and hyperparameter-sensitive behavior. **PonderNet** (Banino et al., 2021) reformulates it probabilistically: a geometric termination distribution over the steps, optimized with an expected loss plus a KL regularization toward a geometric prior. It was not designed for language models or for sequences of latent tokens, but its learned *halting* mechanism is the natural component for making the number of steps dynamic in the implicit-CoT family. Our work performs that graft. An alternative to learned *halting* is an *auxiliary controller or classifier* that predicts the budget from the input or the hidden state; we evaluate the simplest version of that family (*upfront* prediction from the prompt) in Section 4, with a negative result.

## 3 Preliminaries: the latent loop and the experimental protocol

**The fixed-$K$ latent loop.** All three studies extend the same latent-CoT mechanism. Starting from the prefix of question embeddings $U^{(0)} = (e(x_1), \ldots, e(x_T))$, the model runs $K$ reasoning steps; at each step $k = 1, \ldots, K$ it takes the last-layer hidden state at the last position and feeds it back as the next input:

$$z_k = H_\theta\left(U^{(k-1)}\right), \qquad U^{(k)} = U^{(k-1)} \oplus z_k, \tag{1}$$

where $\oplus$ concatenates along the time axis. After $K$ steps, the model switches to decoding over the vocabulary and produces the answer. The crucial point is that $K$ is fixed in advance and identical for every input: it is the rigidity that adaptive *halting* (Section 5) makes dynamic. The three studies share this loop, the data, and the backbone; they differ only in the evaluation protocol, which each one states separately.

**Data and backbone.** All experiments use GSM8k (Cobbe et al., 2021) with the augmented step-by-step internalization annotations of Deng et al. (2024) (GSM8k-Aug), and a GPT-2 (124M) backbone (Radford et al., 2019).[1] *Upfront* prediction (oracle sweep and classifier) works on GSM8k training examples with an internal validation split, whereas adaptive *halting* and the $c$ axis report on the same GSM8k test set (the 1319 held-out examples). The figures for the latter two are therefore directly comparable to each other; those of

[1]The distributions of the number of reasoning steps differ across splits: validation and test examples have, on average, fewer annotated steps than training examples.

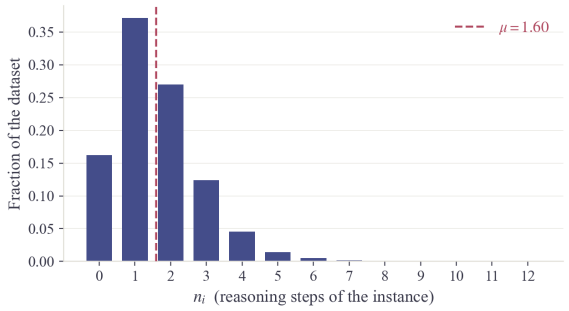


Figure 1: Distribution of the number of annotated reasoning steps $n_i$ in GSM8k-Aug (mean $\mu = 1.59$; more than half of the examples have $n_i \leq 1$).

*upfront* prediction, measured on training with internal validation, are not. Each study is, in any case, internally consistent.

## 4 Anticipating the budget: *upfront* prediction from the prompt

Before building an adaptive mechanism it is worth verifying that there is something to adapt. This first study attacks the $K$ axis in the most direct way: an *oracle sweep* that measures how much budget each instance really needs, and an auxiliary classifier that tries to predict it in advance from the prompt, following the pipeline *prompt* $\rightarrow$ *classifier* $\rightarrow \hat{k} \rightarrow$ *the model reasons with* $\hat{k}$ *steps*. The whole study is evaluated on GSM8k training examples with an internal validation split, not on the held-out test set of adaptive *halting* (Section 5).

### 4.1 Oracle sweep: the premise is confirmed

We run CODI repeatedly over the same examples, varying the number of latent steps $k \in \{1, \ldots, 8\}$, and for each instance we define $k^*$ as the *smallest* $k$ that reaches its best score (the minimal sufficient budget). Over $n$=7473 GSM8k training examples, Table 1 shows two facts. First, the budget **matters**: aggregate accuracy rises 23 points between $k$=1 and $k$≈5–6 and then saturates (in 97.4% of the examples the output changes as $k$ varies). Second, the optimal budget **varies per instance** and is strongly imbalanced: 66% of the examples reach their best answer already at $k$=1, while a long tail needs 2 to 8 steps. There exists, then, the structure that an adaptive mechanism can exploit: most instances have budget to spare and a minority needs all of it.

| $k$ | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|---|---|---|---|---|---|---|---|---|
| Accuracy | .41 | .43 | .48 | .58 | .64 | **.64** | .64 | .63 |
| $k^*$ (%) | **66.1** | 16.6 | 5.3 | 6.0 | 4.3 | 1.0 | 0.4 | 0.3 |

Table 1: Oracle sweep over CODI ($n$=7473 GSM8k training examples, $k_{\max}$=8). Top row: aggregate accuracy when forcing $k$ steps for all instances. Bottom row: distribution of the minimal optimal budget $k^*$ per instance. Accuracy saturates at $k$≈5–6 and $k^*$ is strongly imbalanced toward $k$=1.

| **Model** | **Accuracy** | **F1-macro** |
|---|---|---|
| Majority (always $k$=1) | 0.661 | 0.099 |
| Logistic reg. (balanced) | 0.210 | 0.111 |
| Random forest (balanced) | 0.661 | 0.099 |

Table 2: *Upfront* classifier of $k^*$ over frozen embeddings (internal validation, $n$=1495). No model beats the majority class: the prompt representation carries no usable $k^*$ signal above the base rate.

### 4.2 The *upfront* classifier: a negative result

The simplest realization of adaptivity uses those $k^*$ labels to train a classifier that predicts the budget *before* reasoning. Our first version, deliberately simple, classifies $k^* \in \{1, \ldots, 8\}$ over frozen sentence embeddings (all-MiniLM-L6-v2, 384 dimensions; Reimers and Gurevych, 2019), with a stratified 5978/1495 split. Table 2 summarizes the internal validation: the balanced *random forest* **collapses exactly to the majority baseline** (it predicts $k$=1 for all 1495 examples), and the balanced logistic regression spreads its predictions but gains only +0.012 macro-F1 at the cost of sinking accuracy to 0.21: it disperses, it does not discriminate.

### 4.3 From predicting to deciding on the fly

The negative result is informative because it points to structural limitations of the *upfront* approach, not just of its first implementation. (i) The classifier must infer difficulty by looking *only at the prompt*, before the main model starts reasoning and without access to its hidden states; the signal it needs may simply not be present in a static representation of the text. (ii) Building the labels costs $k_{\max}$ model passes per instance, and $k^*$ inherits the imbalance and the noise of the metric used to score it. (iii) The decision is made only once: if $\hat{k}$ falls short, the system cannot correct itself on the fly. Adaptive *halting* (Section 5) dissolves all three problems: it decides *during* reasoning by observing the latent states themselves, needs no $k^*$ labels, and is optimized directly by the task signal.

## 5 Deciding on the fly: PonderNet-style probabilistic *halting*

Instead of fixing $K$ or predicting it before reasoning (*upfront* prediction, Section 4), we decide the stopping point *during* reasoning: we graft PonderNet's probabilistic *halting* onto the fixed-$K$ continuous-thought loop (Equation 1), so that the model learns how many latent steps to use per instance by observing its own states. Figure 2 previews the central result: the adaptive model matches the fixed-$K$ baseline accuracy while spending nearly half the compute.

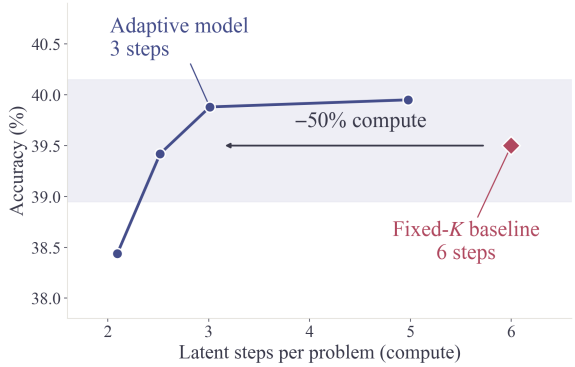


Figure 2: Central result of adaptive *halting*. The adaptive model (M2) reaches the fixed-$K$ baseline accuracy ($\approx 39.5\%$) with $\sim 3$ latent steps per problem, half of the baseline's 6 steps. Each blue point is a different stopping threshold of the same model; the diamond is the baseline. Held-out test ($n$=1319).

### 5.1 Adaptive halting

We add a single linear *halting* head, `halt_head` : $\mathbb{R}^d \to \mathbb{R}$, whose bias is initialized at $-2.0$ (so that the initial halting probability is $\sigma(-2) \approx 0.12$ and the model uses many steps at the start of training). At each latent step $k$ it defines a *conditional halting probability*

$$\lambda_k = \sigma(\texttt{halt\_head}(h_k)), \tag{2}$$

the probability of stopping at step $k$ given that it did not stop earlier. Unlike fixed $K$, we decode a candidate answer after **every** prefix $k = 1, \ldots, K$, which allows building PonderNet's geometric *termination distribution*:

$$p_k = \left( \prod_{j<k} (1 - \lambda_j) \right) \lambda_k. \tag{3}$$

So that the distribution sums to 1 without losing mass, we impose an *absorbing boundary* at $K$ (equivalent to forcing $\lambda_K \to 1$): the last entry receives all the remaining mass $p_K = \prod_{j<K}(1-\lambda_j)$, and it is then renormalized.

### 5.2 Training objective

The objective combines three lineages. From PonderNet, the *expected answer loss* and the KL regularization; from SIM-CoT, the per-step supervision; from CODI, the distillation:

$$\mathcal{L} = \mathcal{L}_{\text{ponder}} + \beta_{\text{step}}\, \mathcal{L}_{\text{step}} + \gamma\, \text{KL}_{\text{geom}} + \mathcal{L}_{\text{distill}} + \mathcal{L}_{\text{ref}}. \tag{4}$$

The main term weights each prefix's answer cross-entropy by its halting probability,

$$\mathcal{L}_{\text{ponder}} = \sum_{k=1}^{K} p_k\, \mathcal{L}_{\text{ans}}^{(k)}, \tag{5}$$

so that the model learns to make good the answers of the prefixes to which it assigns mass. $\mathcal{L}_{\text{step}}$ is the reconstruction of SIM-CoT's auxiliary decoder (with $\beta_{\text{step}} = 1.0$; the decoder is discarded at inference). $\text{KL}_{\text{geom}}$ is the KL divergence of the termination distribution $p_k$ toward a *truncated geometric prior* $q$ with mean `geom_mean`; with $g = 1/\texttt{geom\_mean}$, $q_k \propto (1-g)^k\, g$ with the same absorbing boundary at $K$. The terms $\mathcal{L}_{\text{distill}}$ (SmoothL1 alignment of teacher–student hidden states) and $\mathcal{L}_{\text{ref}}$ (cross-entropy of the teacher task) come from CODI. By default $\gamma = 0.01$ and $\texttt{geom\_mean} = 3.0$. The weight $\gamma$ is the causal lever that regulates how much the compute is shortened: it raises the pressure to stop earlier.

### 5.3 Per-instance geometric prior

A *global* geometric prior asks the same expected budget of every problem. We replace it with a **per-instance** prior that anchors the budget to the problem's number of operations $n_i$ through an affine mapping:

$$\texttt{geom\_mean}_i = \text{clamp}\big(\alpha\, n_i + \beta,\ \beta,\ K_{\max}\big). \tag{6}$$

The affine form (rather than the identity $\texttt{geom\_mean}_i = n_i$) is necessary because of the skew of the $n_i$ distribution in GSM8k-Aug (Figure 1): more than half of the examples have $n_i \leq 1$, which under the identity would fall into a degenerate point-mass prior ($g = 1$), whose KL dominates the task loss. The shift $\beta$ lifts all examples off that floor.

| Threshold | M0 $\alpha 1.0, \gamma .05$ | M1 $\alpha 1.0, \gamma .10$ | M2 $\alpha 0.6, \gamma .10$ |
|---|---|---|---|
| 0.3 | 39.1 @ 3.34 | 39.1 @ 2.60 | 38.4 @ **2.10** |
| 0.4 | 39.6 @ 3.86 | 39.4 @ 3.16 | 39.4 @ **2.52** |
| 0.5 | 40.0 @ 4.46 | 39.4 @ 3.73 | **39.9 @ 3.01** |
| 0.8 | 40.2 @ 6.97 | 39.7 @ 6.39 | 40.0 @ 4.98 |

Table 3: Accuracy–compute frontier. Each cell is *accuracy (%) @ average latent steps*, on the held-out test set ($n = 1319$), *greedy*, `batch_size`$= 1$. Accuracy is on par across the three configurations ($\sim$ 39–40%); raising $\gamma$ (M1) and lowering $\alpha$ (M2) trim the steps at parity accuracy. (M0/M1/M2 are the same model at three configurations of $\gamma$ and prior shape.)

### 5.4 Inference

At inference we accumulate halting mass step by step and cut the latent loop as soon as the accumulated mass crosses a threshold (by default 0.5); $K_{\max}$ = `max_latent_steps` is a hard cap. The answer is decoded from the prefix at which the instance stopped. A fidelity detail: since the latent steps are shared within a batch but each example stops at a different step, a faithful evaluation requires `batch_size`$= 1$ (or cutting the KV-cache row by row at each example's stopping step); otherwise an example that stops early would be decoded from a longer prefix than its step count reports. All the numbers in this work use the faithful variant. The checkpoint (epoch) and the stopping threshold were chosen on a separate validation set of 500 examples, never on the held-out test set on which this section's figures are reported.

### 5.5 The accuracy–compute frontier

The central result of this study is that the adaptive model traces an accuracy–steps **frontier** that fixed $K$ cannot: by varying $\gamma$ (which makes the geometric prior "bite") and the shape of the per-instance prior, we move the operating point monotonically. Table 3 compares, per stopping threshold, three operating points that differ only in the strength $\gamma$ of the KL regularizer and the slope $\alpha$ of the per-instance prior: **M0** (the weak reference prior), **M1** (which hardens $\gamma$), and **M2** (which additionally lowers $\alpha$ to trim the budget tail on hard problems). The exact values of each configuration are in the table.

On test, accuracy is essentially flat ($\sim$ 39–40%) across configurations and thresholds, so the choice of operating point is one of *compute budget*. The minimum-compute point at parity is **M2** at threshold 0.5: **39.9**% with **3.01** steps, matching the fixed-$K$ baseline (39.5%) with **half** the steps, and $-33\%$ steps relative to M0 at the same threshold (at $-0.1$ pp). Lowering the threshold to 0.4/0.3 compresses the compute even further (M2: 2.52/2.10 steps) with small accuracy drops. A $\gamma$ that is too low (0.05, M0) makes the prior not bite; raising it to 0.10 (M1/M2) shifts the whole frontier to the left. Figure 3 visualizes this. Accuracy parity also holds *bin by bin* of difficulty (Figure 4): the compute cut is not paid for in any regime, neither on easy nor on hard problems.

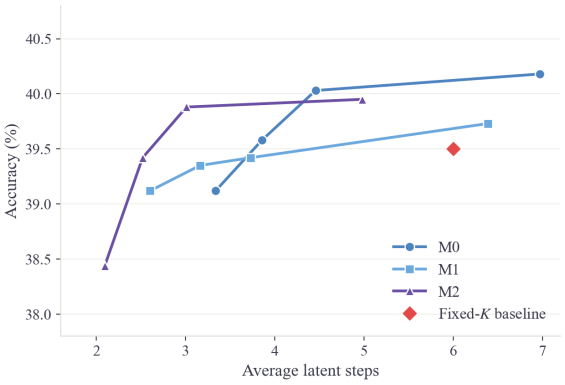


Figure 3: Accuracy–compute frontier. Raising $\gamma$ shifts the frontier to the left (fewer steps at equal accuracy); lowering the slope $\alpha$ of the per-instance prior trims the budget tail even further. M2 reaches 39.9% with 3.01 steps, 33% fewer than M0 at the same threshold (40.0% at 4.46 steps) and half of the fixed-$K$ baseline (6 steps).

### 5.6 Difficulty tracking

A truly adaptive model should spend more steps on harder problems. We measure this with the Spearman correlation between the number of operations

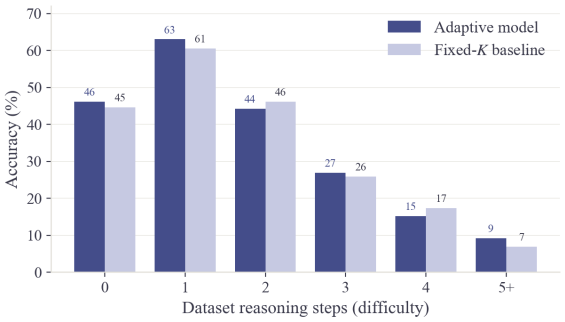


Figure 4: Accuracy by difficulty level (annotated dataset reasoning steps) of the adaptive model (M2, threshold 0.5) against the fixed-$K$ baseline, on held-out test. Parity holds bin by bin: the adaptive model's advantage is in compute, not accuracy, across the whole difficulty scale.

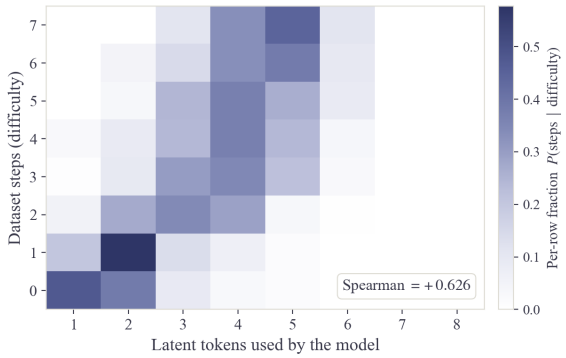


Figure 5: Joint distribution of problem difficulty (annotated dataset steps, rows) and the latent steps the model actually uses (columns), normalized per row. The mass shifts to the right as we descend the rows: the model spends more steps on harder problems (Spearman $+0.626$). M2, threshold $0.5$, held-out test.

$n_{\text{expr}}$ and the steps used. On the held-out test set, the minimum-compute point (M2, threshold $0.5$) reaches $+\mathbf{0.626}$ (M0 and M1 reach even higher, $+0.677$ and $+0.675$ respectively), a strong correlation that confirms that the full-scope fine-tuning with a per-instance prior and $K_{\max} = 12$ allocates compute by difficulty. Figure 5 shows it directly: the joint distribution of difficulty and steps used concentrates its mass along the diagonal.

The effect is interpretable at the difficulty-group level (Figure 6): in that configuration (M2, test, threshold $0.5$), the average steps scale from $1.72$ (0-operation problems) to $2.11$, $2.99$, $3.77$, $3.81$, and $\sim$ $4.1$–$4.6$ (5-or-more-operation problems). Easy problems stop around two steps and hard ones around four and a half: the model allocates compute by difficulty, not by a uniform budget.

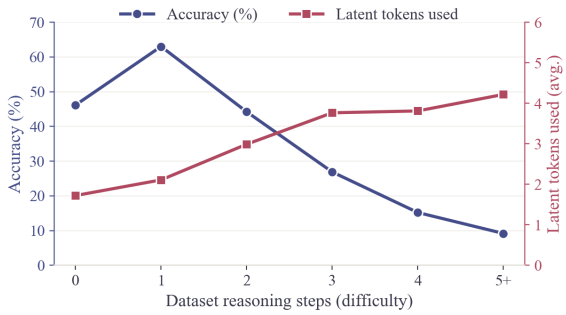


Figure 6: Average latent steps used (red, right axis) and accuracy (blue, left axis) by difficulty level, for the adaptive model (M2, threshold $0.5$) on held-out test. The steps used grow monotonically with difficulty while accuracy falls: compute is allocated where the problem demands it.

### 5.7 Toward a fair from-scratch comparison

The previous experiments start from a warm-start of the full CODI checkpoint (backbone, LoRA, projection, and decoder, the product of a CODI training we did not run), and then fine-tune for only five epochs. A reviewer may rightly ask whether the $\approx$ $40\%$ comes from the adaptive *method* or from the inherited checkpoint. To isolate this, we replicate **M2** (the minimum-compute configuration) from a *vanilla* GPT-2: the auxiliary decoder is also trained (a new `full_dec` *scope*), over the full GSM8k-Aug, $40$ epochs. We record a necessary recipe correction: the original work's $3 \times 10^{-3}$ rate diverges in `bf16` (the gradient norm explodes and `NaN` appears); $1 \times 10^{-3}$ with $0.05$ *warmup* is stable. At the time of submission of this work, the from-scratch model is **still training** (moved to an A100 GPU because of the cost of the sequential $K = 12$ loop). The goal is to report three comparable points (fixed-$K$ baseline, adaptive with warm-start, and adaptive from scratch) so that the gap quantifies the contribution of the warm-start versus that of the method.

## 6 The second budget axis: vectors per step

The latent budget has a second axis: how many vectors $c$ represent each reasoning step. Coconut uses a fixed $c$=2; the CODI checkpoint we start from, $c$=1. This third study makes that number variable per step: each step is built as a block of up to $M$ sub-vectors generated one by one, SIM-CoT's auxiliary decoder reconstructs the step's text from the accumulated block, and a lightweight MLP *distills* that reconstruction loss $\mathcal{L}_{\text{step}}$ from the hidden state ($\hat{\mathcal{L}} = \text{MLP}(h)$, SmoothL1 loss). Unlike the decoder, the MLP survives inference: after each sub-vector $\hat{\mathcal{L}}$ is evaluated and, when it converges ($|\hat{\mathcal{L}}_j - \hat{\mathcal{L}}_{j-1}| < \varepsilon$), the step is considered semantically mature and the model advances to the next one. An optional *ponder penalty* penalizes extra sub-vectors when the step already looks mature.

**The mechanism works; the axis has no headroom.** Trained with warm-start over the atomic segmentation of GSM8k-Aug (one arithmetic operation per step) and evaluated on the full test set ($n$=1319, `batch_size`= 1), adaptive *halting* trims between $16\%$ and $33\%$ of the latent vectors with no accuracy cost ($\approx$ $39.5$–$39.9\%$, competitive with the baseline). But at matched budget it **beats nei-**

| **Init × granularity** | $c$=1 | $c$=2 | $c$=3 | $\Delta_{2\to3}$ |
|---|---|---|---|---|
| warm + atomic | 27.5 | 39.4 | 39.9 | +0.5 |
| warm + coarse | 25.5 | 36.6 | **40.3** | +3.7 |
| *cold* + coarse | 5.3 | 5.5 | 5.5 | n/a |

Table 4: Accuracy (%) with forced fixed $c$, by initialization and segmentation granularity (GSM8k test, $n$=1319). With atomic steps the curve saturates at $c$=2; with coarse steps (2–3 operations, variable density) the third vector contributes again. From scratch the model collapses: the latents remain inert.

**ther random *halting* nor a fixed** $c$: the accuracy curve against fixed $c$ (Table 4, first row) rises +11.9 points from $c$=1 to $c$=2 and only +0.5 from $c$=2 to $c$=3. It saturates hard at $c$=2, almost uniformly across instances: the MLP does not even assign more vectors to the problems the model gets wrong (10.28 vectors on average for correct vs. 10.01 for incorrect). The $c$ axis matters, but its *per-instance variance* is almost nil on this task: the simple optimum is a fixed $c$=2, and there is no clever adaptivity to exploit.

**The negative is about granularity, not the mechanism.** Two controls bound the conclusion. First, retraining *from scratch* (vanilla GPT-2, fresh decoder) collapses to near chance ($\approx$ 5.5% for all $c$): the model learns the answer format but the latents remain inert (decoding after each prefix gives a flat curve), i.e., latent reasoning does not start from scratch under this objective and the warm-start was not a bias but a structural piece. Second (and more interesting), re-segmenting the steps in a *coarse* way (grouping 2–3 operations per step, with variable density) over the warm model revives the signal: the jump $c$=2 $\to$ $c$=3 goes from +0.5 to +3.7 points (Table 4), and adaptive *halting* comes to beat random *halting* by +3.6 points ($\approx 2.7\sigma$; 38.8% with 7.2 vectors versus 35.2% with 6.1), something that did not happen in the atomic regime, although the gain over assigning everyone the same budget is modest (+0.6 to +1.0). The combined reading: in GSM8k-Aug each atomic step is a trivial operation that two vectors saturate; the task's variance lives in the *number* of steps, not in their internal complexity. This bounds the design space and reinforces the choice of the $K$ axis as the one with real dynamics.

## 7 Discussion

We interpret the learned *halting* as an *early-exit* mechanism that avoids the collapse-prone high-$K$ regime. SIM-CoT (Wei et al., 2025) documents that late latents homogenize and merely "repeat" the final prediction once the answer is reached; stopping when the answer is already encoded preserves the diversity among latents and acts as a regularizer of the number of steps, without treating it as a hyperparameter to tune. That accuracy stays within the noise is consistent with this reading: the method does not aim to solve problems that fixed $K$ cannot, but to solve the same ones while spending compute proportional to difficulty. The frontier in Figure 3 is the operational formulation of that idea.

Read together, the three studies delimit where adaptivity appears in this family: the oracle sweep (Section 4) shows that the optimal budget varies per instance; that *upfront* prediction does not beat the base rate shows that this variation is not readable from a static representation of the prompt; and the exploration of the $c$ axis (Section 6) shows that it does not live in the internal density of each step either. What remains is the *number* of steps, decided during reasoning (Section 5), as the axis with exploitable dynamics.

## Limitations

Several caveats bound our conclusions. First, even with 1319 test examples, accuracy differences of fractions of a point are within the sampling noise; our accuracy claims are of *parity*, not of improvement. The second and third share a root: our bounded compute budget. Because of it we evaluate a single backbone (GPT-2) and a single task (GSM8k), so we do not know how adaptivity scales to larger models or other reasoning domains; and because of that same constraint the fair from-scratch comparison (Section 5.7), whose sequential $K$=12 loop is particularly costly, is still training at submission time and has no final number. Fourth, as we stated when presenting each study (Section 3), *upfront* prediction was evaluated on training examples with internal validation, whereas adaptive *halting* and the $c$ axis share the GSM8k test set (1319 held-out examples): the *upfront* prediction figures are not directly comparable with the others, and the *end-to-end* comparison between predicting $k$ *upfront* and learned *halting* remained pending. Finally, our inference stopping criterion is a deterministic threshold on the accumulated mass, which differs from the Bernoulli sampling of PonderNet's original formulation (Banino et al., 2021); we do not compare the two regimes.

## 8 Conclusion

We studied how to make the latent reasoning budget instance-adaptive, exploring its two axes in three studies. *Upfront* prediction confirmed the premise with an oracle sweep (the optimal step budget exists, varies per instance, and saturates), but left an informative negative result: anticipating that budget from frozen prompt embeddings does not beat the base rate. Adaptive *halting*, the one that does work, decides *during* reasoning: we grafted PonderNet's probabilistic *halting* onto the CODI/SIM-CoT stack and, on the held-out test set, the model matches the fixed-$K$ baseline accuracy ($\approx 39.5\%$ on GSM8k with GPT-2) while cutting nearly half ($\sim 50\%$) of the latent steps, with strong difficulty tracking (Spearman $+0.626$). The $c$-axis study showed that this other axis, vectors per step, has almost no exploitable variance on GSM8k-Aug. The contribution is not higher accuracy but **compute adaptivity**: the characterization of the accuracy–steps frontier that the implicit-CoT family, tied to a fixed $K$, never reported.

## References


Andrea Banino, Jan Balaguer, and Charles Blundell. 2021. PonderNet: Learning to ponder. In *8th ICML Workshop on Automated Machine Learning (AutoML)*.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*.

Yuntian Deng, Yejin Choi, and Stuart Shieber. 2024. From explicit CoT to implicit CoT: Learning to internalize CoT step by step. *arXiv preprint arXiv:2405.14838*.

Alex Graves. 2016. Adaptive computation time for recurrent neural networks. *arXiv preprint arXiv:1603.08983*.

Shibo Hao, Sainbayar Sukhbaatar, DiJia Su, Xian Li, Zhiting Hu, Jason Weston, and Yuandong Tian. 2024. Training large language models to reason in a continuous latent space. *arXiv preprint arXiv:2412.06769*.

Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. 2019. Language models are unsupervised multitask learners. *OpenAI Technical Report*.

Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence embeddings using siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Zhenyi Shen, Hanqi Yan, Linhai Zhang, Zhanghao Hu, Yali Du, and Yulan He. 2025. CODI: Compressing chain-of-thought into continuous space via self-distillation. *arXiv preprint arXiv:2502.21074*.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc V. Le, and Denny Zhou. 2022. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Xilin Wei, Xiaoran Liu, Yuhang Zang, Xiaoyi Dong, Yuhang Cao, Jiaqi Wang, Xipeng Qiu, and Dahua Lin. 2025. SIM-CoT: Supervised implicit chain-of-thought. *arXiv preprint arXiv:2509.20317*.